

Sesión 6

Navegación entre pantallas

Menú → Juego → Menú sin recargar la página

■ Duración 1 hora	■ Sesión #6 de 18	■ Fase 2 — Sistema base	■ Resultado Navegación funcional
-----------------------------	-----------------------------	-----------------------------------	--

● 1. Idea central de la sesión

Una app no es un montón de páginas web. Es un sistema que decide qué mostrar y cuándo, sin recargar nada. La navegación es lógica, no magia del navegador.

Al final de esta sesión debe quedar claro que:

- Navegar entre pantallas es cambiar el estado, no cargar HTML nuevo
- La lógica de qué se ve depende del código, no del botón de atrás del navegador
- Una sola página puede parecer una app completa si el estado está bien gestionado

● 2. Conexión con conocimientos previos

■ En Scratch	■■ En nuestra app
En Scratch cambiabas de escenario	En nuestra app cambiamos el estado
Había botones para ir a otra pantalla	Los botones disparan funciones JS
Scratch recargaba la escena entera	JS muestra/oculta elementos sin recargar
No controlabas qué datos viajaban	Nosotras decidimos qué datos se guardan

Lo que ya sabes de la Sesión 5:

- El estado vive en un objeto JavaScript: `estado = { pantallaActual: 'menu', ... }`
- Cambiar el estado es llamar a una función, no tocar variables sueltas
- El patrón es: evento → función → estado cambia → UI se actualiza

● 3. Explicación clara del concepto

¿Qué es la navegación en una app?

Cuando usas una app en tu móvil y vas de la pantalla de inicio a un juego, no se descarga una página nueva. El teléfono simplemente decide qué mostrar y qué esconder. Eso es **navegación basada en estado**.

En la web clásica, cambiar de página significa que el navegador pide un archivo nuevo al servidor, se pierde todo lo que había cargado y empieza desde cero.

En nuestra app queremos algo diferente:

- Hay un solo archivo HTML
- JavaScript decide qué "sección" es visible
- El estado recuerda dónde estamos

■ Palabra clave: SPA (Single Page Application)

Una sola página que se comporta como varias. El truco es JavaScript, no más archivos HTML.

El mecanismo concreto

Tenemos varias secciones en el HTML, cada una con un id. Por defecto, todas menos una están ocultas. Cuando el usuario pulsa un botón, JavaScript:

1. Actualiza el estado: `estado.pantallaActual = 'juego'`
2. Oculta la pantalla anterior
3. Muestra la pantalla nueva

● 4. Diseño antes de código

¿Qué partes tiene el sistema de navegación?

Parte del sistema	Responsabilidad	Por qué existe
Estado de navegación	<code>pantallaActual</code>	Saber dónde está la usuaria en cada momento
Función <code>navegar()</code>	Cambia <code>pantallaActual</code> y actualiza la UI	Único punto de control de la navegación
Renderizador de pantalla	Lee el estado, muestra/oculta secciones	La UI siempre refleja el estado, nunca al revés
Botones y eventos	Llaman a <code>navegar()</code> con un parámetro	Conectan la acción del usuario con la lógica
Secciones HTML	Divs con id único por pantalla	La estructura que la lógica controla

Diagrama de flujo

Usuaria pulsa 'Jugar'

- botón llama a navegar('juego')
- función actualiza estado.pantallaActual = 'juego'
- función oculta pantalla-menu
- función muestra pantalla-juego

■ Usuaria ve el juego

■ Pregunta de diseño importante

¿Dónde debería vivir la función navegar()?

- A) Dentro del botón (onclick en HTML)
- B) En app.js como función separada
- C) En estado.js junto al estado

Piénsalo antes de seguir. La respuesta correcta tiene que ver con quién debe tener esa responsabilidad.

● 5. Ejemplo aplicado al proyecto

Las pantallas de nuestra app

Valor de pantallaActual	Qué se muestra
menu	Seleccionar qué juego jugar
juego-tic-tac-toe	Jugar al Tic Tac Toe
juego-ahorcado	Jugar al Ahorcado
resultado	Ver puntuación y volver

Código mínimo para entender el concepto

```
// estado.js
const estado = {
  pantallaActual: 'menu',
  juegoActivo: null
};

// app.js
function navegar(pantalla) {
  estado.pantallaActual = pantalla;
  actualizarUI();
}

function actualizarUI() {
  // Ocultar todas las pantallas
  document.querySelectorAll('.pantalla').forEach(p => {
    p.style.display = 'none';
  });
  // Mostrar la pantalla correcta
  document.getElementById('pantalla-' + estado.pantallaActual)
    .style.display = 'block';
}

// Ejemplo de uso:
// navegar('juego-tic-tac-toe');
// navegar('menu');
```

Lo importante no es memorizar el código. Hay que notar:

- Una función hace una sola cosa (navegar solo navega)
- El estado es la fuente de verdad
- La UI se adapta al estado, no al revés
- Si queremos añadir una pantalla nueva, solo añadimos un div y un valor al estado

● 6. Uso guiado de agentes de IA

Qué pedirle a un agente esta semana

■ BUENA pregunta para la IA:

"Tengo esta estructura HTML con 3 divs que representan pantallas.
Tengo una función navegar(pantalla) en JS.
Genera la función actualizarUI() que lee estado.pantallaActual
y muestra/oculta los divs correctos.
Explícame cada línea."

■ MALA pregunta para la IA:

"Hazme la navegación de mi app."

Qué revisar obligatoriamente

- ¿La función navegar() modifica el estado ANTES de actualizar la UI?
- ¿La UI solo lee el estado, nunca lo modifica directamente?

- ¿Hay alguna variable de pantalla suelta fuera del objeto estado?
- ¿Los ids en HTML coinciden exactamente con los valores del estado?
- ¿La IA ha puesto lógica de juego dentro de la función de navegación? (eso está mal)

Errores típicos que comete la IA

Error común de la IA	Cómo se manifiesta	Cómo corregirlo
Mezcla navegación con lógica de juego	navegar() también inicializa el juego, resetea el tablero...	Pedir que las separe: una función que solo navega, otra que inicializa
Variables globales sueltas	let pantallaActual en lugar de estado.pantallaActual	Preguntar: '¿Usa el objeto estado centralizado?'
Múltiples formas de navegar	display:none, remove/add class e innerHTML mezclados	Pedir consistencia: un solo mecanismo
Hardcodea nombres de pantalla	if (pantalla=='menu') {...} if (pantalla=='juego') {...}	Preguntar: '¿Cómo sin un if por cada pantalla?'

● 7. Preguntas de pensamiento crítico

Estas preguntas no tienen respuesta en Google. Requieren pensar.

1. Si el estado tiene pantallaActual = 'juego' pero visualmente se ve el menú, ¿qué ha salido mal? ¿Quién tiene razón, el estado o la pantalla?
2. ¿Qué pasa si la usuaria pulsa el botón 'Atrás' del navegador? ¿Nuestra app lo sabe? ¿Debería importarnos?
3. ¿Por qué es mejor que navegar() sea una función y no que cada botón tenga su propio código?
4. Si mañana añadimos una pantalla de 'configuración', ¿qué habría que cambiar en el sistema? ¿Cuántas líneas?
5. ¿Podría la función navegar() recibir datos además del nombre de la pantalla? ¿Para qué serviría?
6. Un amigo dice: 'Yo lo haría con tres archivos HTML distintos, es más sencillo.' ¿Qué le responderías?

● 8. Mini-reto para 1 hora

Reto: El sistema de navegación real de tu app

Este reto implica decidir, no solo ejecutar. No hay una solución única correcta.

Fase 1 — Diseño (15 min, sin tocar código)

En papel o en comentarios de código, responde:

- ¿Cuántas pantallas necesita tu app ahora mismo?
- ¿Qué información necesita cada pantalla para mostrarse correctamente?
- ¿Qué datos del estado deben guardarse cuando navegas fuera de un juego?

- ¿Qué debería pasar si intentas navegar a una pantalla que no existe?

Fase 2 — Implementación mínima (30 min)

Con ayuda de la IA si quieres, implementa:

- Estructura HTML con al menos 3 pantallas (divs con clase 'pantalla')
- Un objeto estado con pantallaActual
- Una función navegar(pantalla) que actualice el estado y la UI
- Botones en el menú que lleven a cada sección
- Un botón 'Volver al menú' en cada sección

Fase 3 — Revisión crítica (15 min)

Antes de cerrar, responde:

- ¿Hay algún lugar del código donde cambias la UI sin pasar por navegar()?
- ¿Qué pasa si abres la app directamente en la pantalla de un juego? ¿Funciona?
- Si la IA te ha generado código, explica en voz alta qué hace cada función

■ Resultado esperado

Navegación funcional entre menú y al menos una pantalla de juego, con el estado controlando qué se ve.

● 9. Qué NO hacer

■ Error	Por qué es un problema	■ En su lugar
Usar window.location.href	Recarga la página, pierde el estado	Cambiar display en el HTML directamente
Poner toda la lógica en onclick del botón	El HTML acaba lleno de JavaScript, ilegible	Llamar a función desde botón, la lógica en JS
Tener pantallaActual suelta como variable global	El estado se desincroniza fácilmente	Siempre dentro del objeto estado
Ocultar/mostrar con CSS sin actualizar estado	La UI y el estado dejan de sincronizarse	Siempre actualizar estado primero, UI después
Copiar código de la IA sin entenderlo	No sabrás qué hacer cuando falle (y fallará)	Pedir a la IA que explique línea por línea
Función navegar diferente por cada pantalla	Código repetido, difícil de mantener	Una sola navegar() con el destino como parámetro

● Resumen de la sesión

La idea que debe quedar:

Navegar = cambiar el estado + actualizar la UI.

El estado manda. La UI obedece. Siempre en ese orden.

■ Conceptos aprendidos	■■ Herramientas usadas
Navegación basada en estado	pantallaActual en el estado
Función navegar() centralizada	actualizarUI() lee el estado
Patrón: estado → UI	display:none / display:block
SPA (Single Page Application)	ids únicos por pantalla en HTML

Preparación para la Sesión 7: Separación de responsabilidades — UI vs lógica, qué va en cada archivo.